

```
(-map 'square '(2 3 4))
(4 9 16)
```

The ‘-map-when’ function takes three arguments — a predicate function, an application function, and a list. All the elements of the list that match the predicate function are transformed using the application function. If an element does not satisfy the predicate function, the value is unchanged. The ‘even?’ function takes a number as the input and determines if it is an even or odd number. This can be used as the predicate function, along with the square function as the application function to the list (2 3 4) as shown below:

```
(-map-when predicate application list) ;; Syntax
```

```
(defun even? (num) (= 0 (% num 2)))
(-map-when 'even? 'square '(2 3 4))
(4 3 16)
```

The ‘-map-first’ function replaces the first element in the list that matches the predicate function and executes the application function on the element. Similarly, the ‘-map-last’ function replaces the last element in the list that satisfies the predicate function by executing the application function on the element. A couple of examples are given below:

```
(-map-first predicate application list) ;; Syntax
(-map-last predicate application list) ;; Syntax
```

```
(-map-first 'even? 'square '(2 3 4))
(4 3 4)
```

```
(-map-last 'even? 'square '(2 3 4))
(2 3 16)
```

The ‘-annotate’ API applies a function to each element of the list, and pairs the result to the unmodified list. An example is shown below:

```
(-annotate function list) ;; Syntax
```

```
(-annotate 'length '(("a" "b" "c") ("foo" "bar")))
((3 "a" "b" "c") (2 "foo" "bar"))
```

The ‘-splice-list’ function takes three arguments — a predicate function, a new list and the input list. It splices the input list at places where the elements match the predicate function with the new list. In the following example, the input list ‘(x :symbol y)’ is spliced between x and y as the predicate function checks for a keyword, and replaces it with the new list ‘(a b c)’.

```
(-splice-list predicate new-list list) ;; Syntax
```

```
(-splice-list 'keywordp '(a b c) '(x :symbol y))
(x a b c y)
```

The ‘-mapcat’ function takes a function and a list, applies the function to each element of the list, and concatenates the results. For example:

```
(-mapcat function list) ;; Syntax
```

```
(--mapcat (list '* it) '(9 8 7))
(* 9 * 8 * 7)
```

Sublist selection

The ‘-filter’ function returns only those elements of a list that match the predicate function passed to it as an argument:

```
(-filter predicate list) ;; Syntax
```

```
(-filter 'even? '(1 2 3 4))
(2 4)
```

The ‘-remove’ function removes those elements that match the predicate function from the input list, and returns a new list as shown below:

```
(-remove predicate list) ;; Syntax
```

```
(-remove 'even? '(1 2 3 4))
(1 3)
```

If you only want to remove the first element or last element from the input list that matches the predicate function, then you can use the ‘-remove-first’ and ‘-remove-last’ functions respectively. A couple of examples are given below for reference:

```
(-remove-first predicate list) ;; Syntax
(-remove-last predicate list) ;; Syntax
```

```
(-remove-first 'even? '(1 2 3 4))
(1 3 4)
```

```
(-remove-last 'even? '(1 2 3 4 5 6 7 8 9 0))
(1 2 3 4 5 6 7 8 9)
```

The ‘-non-nil’ function returns all the elements from the input list that are not nil. For example:

```
(-non-nil '(1 2 3 nil 4 5 6 nil))
(1 2 3 4 5 6)
```

The ‘-slice’ function takes an input list as an argument and returns all the elements between a ‘from’ and ‘to’ position.